



InferenceChain

A Decentralised Blockchain Native to AI Inference

Quality of Inference Consensus · Proof of Quality of Inference · Sharded BFT

Technical Whitepaper & Vision Document

Research Preview · v0.1 · April 2026

Dimitrios Papaioannou

Aristotle University of Thessaloniki (AUTH) · InferenceChain Research

Table of Contents

1. Abstract
2. Problem Statement
3. Architecture Overview
4. Quality of Inference Consensus (QoI)
5. Proof of Quality of Inference (PoQI) — On-chain Reputation
6. Proof of Model (PoM) — Admission Protocol
7. Sharded BFT Architecture (S-BFT)
8. Block Structure & Transaction Types
9. What Is Already Built
10. Tokenomics — INFER Token
11. Vision & Roadmap
12. Conclusion

1. Abstract

InferenceChain is a novel blockchain protocol designed specifically for decentralised AI inference. Unlike general-purpose blockchains that treat computation as an opaque black box, InferenceChain introduces inference *quality* as a first-class consensus property. Validators do not merely agree on transaction order — they agree on the *correctness* of deep neural network (DNN) outputs.

Three novel primitives drive the protocol: **Quality of Inference Consensus (QoI)**, a PBFT-derived algorithm that uses cosine similarity between inference vectors to reach Byzantine-fault-tolerant agreement; **Proof of Quality of Inference (PoQI)**, an on-chain reputation system that quantifies each validator's historical accuracy and weights their future influence accordingly; and **Sharded BFT (S-BFT)**, a horizontal scaling layer that partitions the validator set into independent committees to achieve linear throughput growth without sacrificing BFT safety guarantees.

A working testnet implementation — including the full consensus engine, P2P gossip layer, on-chain state machine, model admission protocol, and a live web dashboard — has been developed and is described in this document.

2. Problem Statement

The rise of AI inference as a service has created a critical centralisation risk. Today, virtually all AI inference is performed by a handful of large cloud providers. Users have no way to verify that the model they requested actually ran, that the output has not been tampered with, or that the service provider is not selectively serving degraded results.

Existing blockchain-based approaches fall into two categories, both inadequate:

Approach	Limitation
Optimistic execution	Assume the inference result is correct and only dispute on-chain after the fact. Disputes are slow, expensive, and rely on a trusted arbiter.
Commit-reveal schemes	A node commits to a result hash then reveals it. This prevents copying but provides no quality guarantee — a node can commit a random hash.
Reputation without verification	Systems that track reputation without on-chain verifiable proof of model quality are gameable. A node can report high accuracy it never achieved.
General-purpose BFT	Standard PBFT agrees on values, not on quality. Two nodes producing different (but equally valid) inference outputs would cause consensus to fail even if both are correct.

InferenceChain addresses all four limitations simultaneously. Consensus is reached on inference *quality*, verified by mathematical comparison (cosine similarity). Reputation is an on-chain quantity, updated deterministically by protocol rules. Admission requires cryptographic proof of model capability

before a node may participate.

3. Architecture Overview

InferenceChain separates concerns cleanly across four layers:

Layer	Responsibility
Network Layer	P2P gossip over WebSocket connections. Each node maintains persistent connections to peers. Messages are typed (CONSENSUS, TRANSACTION, BLOCK, PEER_DISCOVERY) and routed accordingly.
Consensus Layer	Two concurrent consensus protocols. Qol consensus runs when an INFERENCE_REQUEST transaction enters the mempool. PoS consensus runs for all other (simple) transaction blocks.
State Layer	A single ChainState object is the authoritative source of truth — balances, stakes, reputations, registered nodes, inference log, contract state. All reads bypass any off-chain registry.
Application Layer	FastAPI REST + WebSocket API served on port 8000. A compiled React dashboard is served at /ui. All data shown in the UI is read directly from chain state.

4. Quality of Inference Consensus (Qol)

Qol is a three-phase Byzantine-fault-tolerant consensus protocol derived from PBFT, adapted to agree not on an arbitrary value but on the *quality* of a DNN inference result. The key insight is that correct DNN inference on the same image produces output probability vectors that are mathematically close — measurable by cosine similarity.

The protocol requires $n \geq 3f + 1$ validators to tolerate f Byzantine nodes.

Phase 1 — PRE-PREPARE

The primary (proposer) receives an INFERENCE_REQUEST containing an image hash. It fetches the image from the requesting client, runs it through its local DNN model, and broadcasts a PRE_PREPARE message containing its output probability vector (softmax over 10 CIFAR-10 classes).

Phase 2 — PREPARE

Each DNN validator independently runs the same image through its own local model. It computes the cosine similarity between its output vector and the primary's vector. If the similarity exceeds the threshold, it broadcasts a PREPARE message with its own vector. The primary collects PREPARE

messages until it has $2f+1$.

Phase 3 — COMMIT

Once $2f+1$ PREPARE messages are collected, the primary broadcasts a COMMIT message. Each validator that has seen $2f+1$ PREPAREs broadcasts its own COMMIT. When a node collects $2f+1$ COMMITs, the round is finalised. A QoI block is appended to the chain containing the consensus result, all signatures, and reputation deltas.

Quality Scoring

After each round, each validator's output vector is compared pairwise against all others. Validators whose vectors lie within the cosine similarity threshold of the majority cluster are marked as honest and receive reputation increases proportional to their similarity score. Outliers (Byzantine or low-quality nodes) are penalised.

5. Proof of Quality of Inference (PoQI)

PoQI is the on-chain reputation system that tracks each validator's cumulative inference quality. It is not a separate consensus protocol — it is a deterministic state transition that occurs at the end of every QoI round.

Reputation parameters (from protocol constants):

Initial reputation	0.0 (all nodes start equal)
Max gain per round	0.5 (REP_CAP_PER_ROUND)
Penalty per Byzantine act	0.3 (REP_PENALTY)
Leader bonus per round	+0.1 on top of quality-proportional gain
Reputation cap	No hard cap — grows unboundedly with consistent quality
Reputation floor	0.0 (cannot go negative)

Reputation serves two functions: it weights the proposer election (higher reputation → higher probability of being selected as primary), and it weights block reward distribution. A validator that consistently produces high-quality inference results earns disproportionately more INFER tokens over time — creating a strong economic incentive for model quality maintenance.

6. Proof of Model (PoM) — Admission Protocol

Before a node may participate in QoI consensus, it must pass the Proof of Model admission protocol. PoM prevents Sybil attacks and ensures that every active DNN validator in the network actually possesses a model that meets minimum quality standards.

Admission flow:

Step 1 — Register	Node submits a <code>NODE_REGISTER_DNN</code> transaction containing: model name, weights SHA-256 hash (on-chain commitment), endpoint URL, dataset ID, architecture JSON, and public key. Node status: <code>PENDING_POM</code> .
Step 2 — Challenge	Protocol generates a <code>MODEL_CHALLENGE</code> transaction with 50 deterministically selected CIFAR-10 test samples. Seed = $\text{SHA-256}(\text{prev_block_hash} + \text{node_address})$ — fully reproducible by any verifier.
Step 3 — Response	Challenged node runs the 50 samples through its model and broadcasts a <code>MODEL_RESPONSE</code> transaction with its predictions within the timeout window (30 seconds).
Step 4 — Verification	Existing DNN validators independently verify the response against known CIFAR-10 test labels. Each broadcasts a <code>MODEL_VERIFY</code> transaction with pass/fail.
Step 5 — Admission	If $\geq 80\%$ accuracy is achieved and enough verifiers agree, a <code>NODE_ADMITTED</code> transaction is committed. Node becomes active. If it fails: <code>NODE_REJECTED</code> — node must re-register.

The weights hash committed in Step 1 binds the node to a specific model. If a node later attempts to serve a different model, the hash mismatch is detectable on-chain by any verifier — providing cryptographic proof of model substitution fraud.

7. Sharded BFT Architecture (S-BFT)

Standard PBFT has $O(n^2)$ message complexity — as the validator set grows, the communication overhead grows quadratically, making it impractical beyond ~100 nodes. S-BFT solves this by electing a small **quorum** of K nodes per inference request, rather than involving the full validator set.

The scaling problem

With N DNN nodes in the network, running QoI across all of them for every inference request is unscalable: $O(N^2)$ message complexity, latency that tracks the slowest node, and most nodes having no familiarity with the requested model or dataset. S-BFT reduces the active committee to $K = 10$ nodes (configurable), achieving $O(K^2)$ message complexity independent of total network size.

Per-request quorum selection

For each inference request, a quorum is elected as follows:

1. Filter eligible DNN validators by model name and dataset ID matching the request.
2. Compute seed = SHA-256(block_hash || request_id) — unique per round, reproducible by any node.
3. Sample K nodes from the filtered pool using the seed — without replacement, deterministically.
4. Sort selected nodes by node_id for canonical ordering.

Quorum assignment is verifiable by any network participant and unpredictable before the inference request arrives — preventing adversaries from pre-positioning nodes.

Quorum parameters

Target quorum size	10 nodes (ideal)
Floor	4 nodes minimum — gives f=1 (requires 3f+1)
Ceiling	19 nodes maximum — gives f=6
Fallback	If filtered pool < 4 nodes, all eligible DNN validators participate

Local BFT parameters

Each quorum operates with its own fault-tolerance parameters, independent of the global chain f:

$$f = (K - 1) // 3$$

$$\text{threshold} = 2f + 1$$

For K=10: f=3, threshold=7. For K=4: f=1, threshold=3. Nodes not elected to the quorum stand by for one block slot — they do not participate and do not interfere. The committed QoI block arrives via P2P gossip and is ingested normally by all nodes.

Quorum integrity at commit

When a QoI block is sealed, the protocol validates that every commit signature originated from a node in the elected quorum. Signatures from outside the quorum are stripped — those nodes receive no reward. The elected quorum node IDs are embedded in the ConsensusOutcome and written to chain in the CONSENSUS_RESULT transaction.

8. Block Structure & Transaction Types

Two block types

QoI Block	Produced after a successful QoI consensus round. Contains exactly one INFERENCE_REQUEST, plus system transactions: INFERENCE_RESPONSE, CONSENSUS_RESULT, REWARD entries for each participating validator, and reputation delta updates. Committed by the QoI state machine.
PoS Block	Produced for all simple (non-inference) transactions. Contains up to 50 simple transactions per block. Proposer is selected by stake × reputation weighting. Committed by the PoS consensus machine with 2f+1 votes.

Transaction taxonomy

Type	Family	Purpose
TOKEN_TRANSFER	Simple	Peer-to-peer INFER token transfer
STAKE / UNSTAKE	Simple	Lock / unlock tokens as validator stake
NODE_REGISTER_DNN	Simple	Register as a DNN validator (triggers PoM)
NODE_REGISTER_POS	Simple	Register as a PoS-only validator
MODEL_RESPONSE	Simple	Node's answer to a PoM challenge
INFERENCE_REQUEST	Inference	Client submits an image for consensus inference
INFERENCE_RESPONSE	System	Protocol records a validator's output vector
CONSENSUS_RESULT	System	Final agreed inference result
REWARD	System	Token minted to a validator for honest participation
SLASH	System	Tokens burned from a validator for Byzantine behaviour
MODEL_CHALLENGE	System	Protocol challenges a pending node's model
MODEL_VERIFY	System	Existing validator verifies a challenge response
NODE_ADMITTED	System	Node passed PoM — becomes active
NODE_REJECTED	System	Node failed PoM — must re-register

9. What Is Already Built

The following components are fully implemented and operational on the local testnet as of April 2026. This is not a paper prototype — all described functionality runs as working Python/React code.

■ Blockchain Core

Full chain with QoI and PoS block types, Merkle root computation, SHA-256 block hashing, chain validation, genesis block with 10M INFER allocation. SQLite-backed persistent storage on by default (data/chain.db, WAL mode).

■ State Machine

ChainState tracks balances, stakes, reputations, registered nodes (DNN & PoS), PoM states, inference log, smart contract state, and a public key registry (pubkeys) for wallet signature verification. All reads are authoritative from chain, no off-chain registries.

■ S-BFT Quorum Selection

Per-request deterministic quorum election (core/consensus/quorum.py). Seed = SHA-256(block_hash + request_id). Filters eligible DNN nodes by model/dataset, samples $K=10$ nodes, computes quorum-local $f=(K-1)/3$. Non-quorum nodes stand by. Quorum integrity enforced at commit — out-of-quorum signatures stripped.

■ QoI Consensus Engine

Full 3-phase PBFT-derived consensus (PRE-PREPARE → PREPARE → COMMIT). View-change protocol for primary failure recovery. Timeout manager. Message scheduler. Async state machine. QoI state machine initialised with quorum-local f .

■ PoS Consensus

Parallel PoS consensus for simple transaction blocks. Stake × reputation weighted proposer election. $2f+1$ vote threshold. Auto-commits in single-node dev mode.

■ Proof of Model (PoM)

Full admission protocol: challenge generation (deterministic seed), response verification against CIFAR-10 ground truth, tally logic, NODE_ADMITTED / NODE_REJECTED system transactions.

■ Wallet & Signature Verification

secp256k1 BIP39 wallet (12-word mnemonic, AES-256-GCM encrypted). Private key never stored in plaintext. Transactions include sender public key; backend registers it in ChainState.pubkeys on first confirmed tx. Subsequent txs verified using the registry. Signature: ECDSA(SHA-256(tx_id), privkey).

■ P2P Gossip Layer

WebSocket-based peer-to-peer network. Message types: CONSENSUS, TRANSACTION, BLOCK, PEER_DISCOVERY. Persistent connections with reconnect logic. Image fetch protocol for inference requests.

■ REST API (FastAPI)

40+ endpoints across /chain, /state, /tx, /p2p, /dashboard, /ws namespaces. Full OpenAPI docs at /docs. WebSocket streams for live consensus events. Faucet endpoint for testnet token distribution.

■ Model Registry

7 CIFAR-10 CNN architectures (ResNet, VGG, DenseNet, WideResNet, MobileNetV2, PreActResNet, PyramidNet). ModelRegistry with SHA-256 weights commitment. Training scripts included.

■ Cryptographic Identity

secp256k1 keys (ecdsa lib on backend, @noble/secp256k1 on frontend). Node identity persisted to JSON keyfiles. Address = SHA-256(pubkey_bytes)[:20] as 40-char hex — identical derivation in Python and JavaScript.

■ Frontend Dashboard

React SPA with 7 pages: Home (live stats, canvas particles, circuit hero), Explorer (clickable blocks/txs/addresses, address history), Validators, Inference, Network, Become a Node (DNN registration wizard), Whitepaper. Full wallet panel: create, backup, recover, import, lock/unlock, send, stake/unstake.

■ Vercel Deployment

Production build configured for Vercel with vercel.json. .env.production sets relative API URLs for same-origin serving.

■ Test Suite

Unit and integration tests covering chain state transitions, QoI consensus rounds, PoM flow, P2P image fetch, dashboard API, transaction serialisation, and wallet pubkey registry.

10. Tokenomics — INFER Token

The INFER token is the native currency of InferenceChain. It serves three distinct roles: **economic incentive** for honest inference quality, **security bond** that makes Byzantine behaviour costly, and **governance weight** for future protocol parameter votes.

Supply parameters

Token name	INFER
Maximum supply	100,000,000 INFER (100M hard cap)
Genesis allocation	10,000,000 INFER (10% — minted at block 0)
Block reward	10.0 INFER per QoI consensus round
Leader bonus	+2.0 INFER for the primary that drove the round
Slash penalty	100.0 INFER burned per confirmed Byzantine act
Min stake — DNN	1,000 INFER
Min stake — PoS	500 INFER

Reward distribution per QoI round

When a QoI round reaches COMMIT, the protocol mints INFER and distributes it as follows:

Recipient	Amount
Primary (proposer)	Base reward $\times (1 + \text{leader_bonus_ratio})$ — higher share for driving consensus
Honest validators	Proportional share of remaining reward, weighted by cosine similarity score
Byzantine validators	Zero reward + 100 INFER slashed from stake
PoS validators	Proportional share of PoS block reward (separate pool, no inference bonus)

Stake economics

Stake serves as a security deposit. A node that is confirmed Byzantine by the protocol loses 100 INFER from its staked balance. If the stake falls below the minimum (1,000 INFER for DNN), the node is automatically deactivated. This creates a direct cost for Byzantine behaviour proportional to the expected value of sustained honest participation — the classical mechanism for making attacks economically irrational.

Reputation × stake interaction

Block reward shares are computed as:

$$\text{reward_share}(i) = (\text{stake}(i) \times \text{reputation}(i)) / \sum (\text{stake}(j) \times \text{reputation}(j))$$

This means a node with twice the stake but half the reputation earns the same as a node with equal stake and reputation — quality and capital are equally weighted. A new node with zero reputation earns nothing until it demonstrates quality, regardless of how much it stakes. This prevents pure capital dominance.

Genesis allocation rationale

10M INFER is pre-minted at genesis and distributed to bootstrap nodes. This provides the initial stake pool necessary for the network to function before any block rewards have accumulated. Genesis nodes are expected to be research nodes operated by the protocol authors — their identity and stakes are committed in the genesis block and auditable by all participants.

Emission schedule

At 10 INFER per Qol block and a 5-second block target, the maximum theoretical emission rate is approximately 6.3M INFER per year ($10 \times 6M \text{ seconds} / 5 \text{ target seconds}$ but reduced by actual block production rate). At this rate, the remaining 90M INFER supply would take ~14 years to emit — providing a long runway of validator incentives. In practice, not every 5-second window produces a Qol block (only when inference requests are pending), so actual emission is demand-driven.

11. Vision & Roadmap

InferenceChain's long-term vision is to become the trust layer for AI inference — the protocol that any application can use to prove that a specific AI model produced a specific output, that the output quality was independently verified, and that the verifiers were economically incentivised to be honest.

This vision positions InferenceChain not as a competitor to AI model providers, but as an infrastructure layer beneath them — analogous to how TCP/IP is not a competitor to web applications but the foundation they rely on.

Phase 1 — Foundation (Current: Testnet v0.1)

- ✓ QoI consensus engine (3-phase PBFT, view change, timeout)
- ✓ PoQI on-chain reputation system
- ✓ Proof of Model admission protocol
- ✓ PoS consensus for simple transactions
- ✓ S-BFT per-request quorum selection (K=10, deterministic seed)
- ✓ secp256k1 wallet — BIP39 mnemonic, AES-256-GCM encryption, send/stake
- ✓ On-chain public key registry + wallet signature verification
- ✓ P2P gossip layer with WebSocket connections
- ✓ Full REST + WebSocket API (40+ endpoints)
- ✓ Live web dashboard (React, 7 pages, chain explorer, wallet panel)
- ✓ DNN validator registration wizard
- ✓ SQLite-backed persistent chain storage (on by default)
- ✓ secp256k1 cryptographic identity (Python + JavaScript)

Phase 2 — Scaling & Security (Q3 2026)

- S-BFT OOD layer — VAE-based Likelihood Regret scoring to prefer domain-familiar nodes in quorum selection
- Signature aggregation — BLS multi-signatures to reduce COMMIT message size
- Peer discovery — full Kademlia DHT for automatic peer finding
- Multi-model support — validators serve multiple DNN architectures
- Slashing evidence proofs — on-chain verifiable proof of Byzantine behaviour
- Light client protocol — verify chain state without full node

Phase 3 — Ecosystem (Q1 2027)

- Smart contracts — Turing-complete VM for inference result consumers
- Cross-chain bridge — export verified inference results to EVM chains

- Model marketplace — on-chain registry of validated model architectures
- DAO governance — INFER-weighted protocol parameter votes
- Mainnet launch — permissionless validator onboarding, full token economy
- SDK — Python/JS libraries for submitting inference requests programmatically

The broader vision

As AI systems become increasingly consequential — in healthcare, law, finance, autonomous systems — the ability to audit and verify AI outputs becomes a regulatory and ethical necessity. InferenceChain provides the cryptographic infrastructure for this audit trail: any inference result committed on-chain can be independently re-verified by anyone, at any time, with mathematical certainty.

The end state is a world where AI inference is not a black box operated by a single trusted party, but a transparent, economically incentivised, and cryptographically auditable process — running on a network of thousands of independent validators, none of which can unilaterally corrupt the result.

12. Conclusion

InferenceChain introduces a fundamentally new class of blockchain protocol — one designed from first principles for AI inference, not adapted from general-purpose designs. The three core innovations (QoI, PoQI, S-BFT) together solve the problem of trustless, verifiable, quality-assured DNN inference at scale.

A complete working implementation has been built, demonstrating that the protocol is not merely theoretical. The testnet runs QoI consensus rounds, admits validators through PoM, tracks on-chain reputation, and serves a full-featured web dashboard — all operating as a coherent system.

The INFER token creates a self-sustaining economic system where quality incentives and security incentives are aligned: the same stake that secures the network proportionally determines reward shares, and reputation — earned only through demonstrated inference quality — amplifies that stake's influence.

InferenceChain is positioned to become the trust infrastructure for the AI era — the protocol that makes AI outputs as verifiable as blockchain transactions.

Dimitrios Papaioannou

InferenceChain Research · Aristotle University of Thessaloniki (AUTH)

April 2026 · Research Preview v0.1